

Introduction

The workshop in the **SQL - data analysis** course is designed to summarize and consolidate the knowledge that has been taught in the course. While solving it, we will be able to see what the tasks of a data analyst in the financial industry can look like.

In our further work we are going to use the [financial](#) database that contains information on loans (either paid in full or not).

It is worth noting that the database uses real data that has been anonymized in order to be made public. It is a collection of financial information from a Czech bank. The dataset contains data for more than 5,300 clients with about one million transactions. What's more, the bank has also released data for nearly 700 loans granted and about 900 credit cards issued.

Note:

All the data that is required to complete the workshop is in the **financial** database. If you want to simplify the query syntax, it is a good idea to write a **USE** clause at this point:

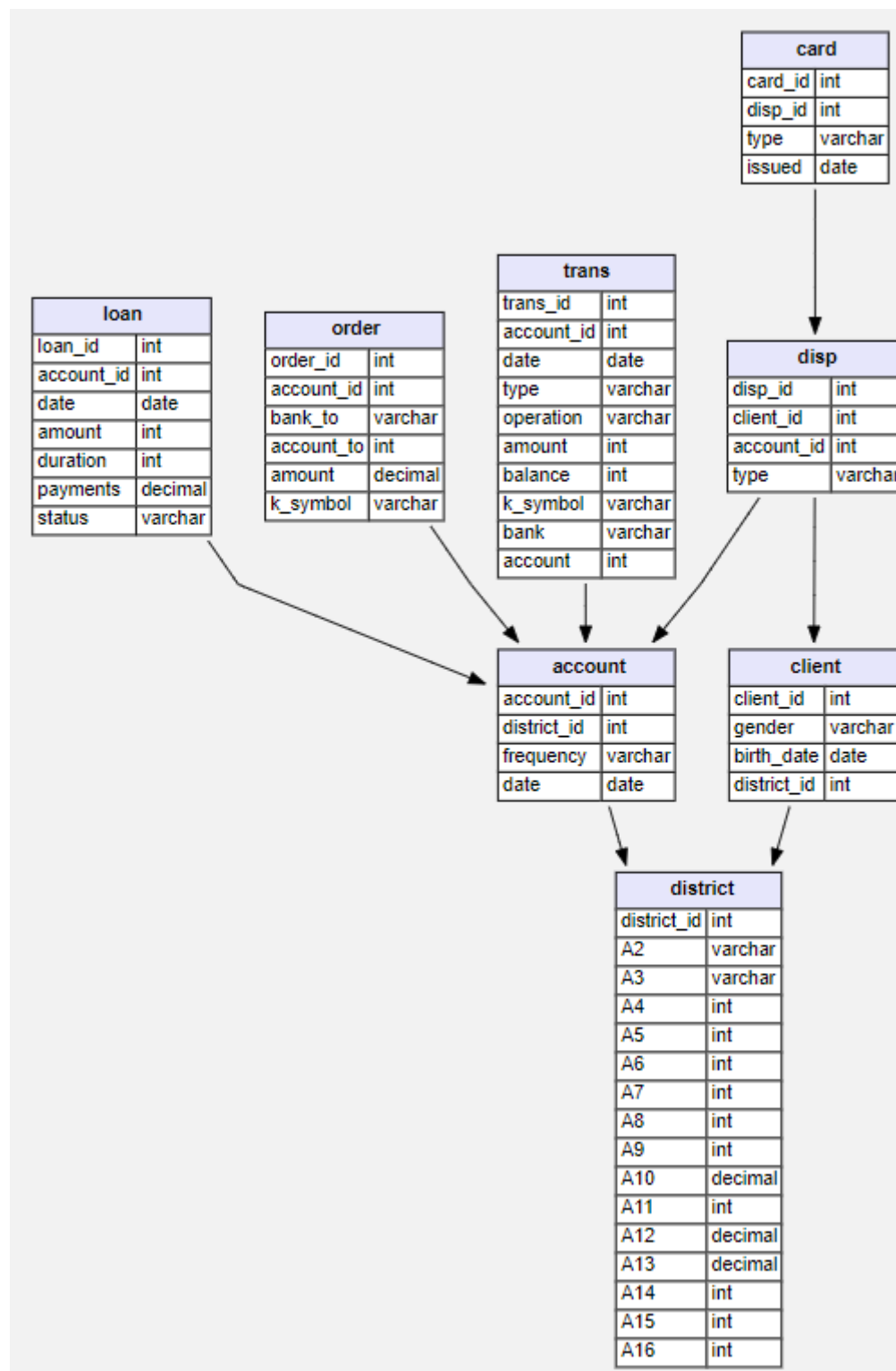
USE financial; -- from this moment on, the default database we use is: financial

1) Structure of financial database

Get familiar with the schema of the database and answer the following questions:

1. What are the primary keys in the individual tables?
2. What relationships do particular pairs of tables have?

Schema



2) History of granted loans

Write a query that prepares a summary of the granted loans in the following dimensions:

- year, quarter, month,
- year, quarter,
- year,
- total.

Display the following information as the result of the summary:

- total amount of loans,
- average loan amount,
- total number of given loans.

3) Loan status

On the database site, we can find information that there are a total of 682 granted loans in the database, of which 606 have been repaid and 76 have not.

Let's assume that we don't have information about which status corresponds to a repaid loan and which does not. In this situation, we need to infer this information from the data.

To do this, write a query to help you answer the question of which statuses represent repaid loans and which represent unpaid loans.

4) Analysis of accounts

Write a query that ranks accounts according to the following criteria:

number of given loans (decreasing),
amount of given loans (decreasing),
average loan amount,

Only **fully paid** loans are considered.

5) Fully paid loans

Find out the balance of repaid loans, divided by client gender.

Additionally, use a method of your choice to check whether the query is correct.

6) Client analysis - part 1

Modifying the queries from the exercise on repaid loans, answer the following questions:

Who has more repaid loans - women or men?

What is the average age of the borrower divided by gender?

Hints:

Save the result of the previously written and then modified query, for example, to a temporary table, and conduct the analysis on it.

You can calculate the age as the difference: 2024 - the year of birth of the borrower.

7) Client analysis - part 2

Make analyses that answer the questions:

which area has the most clients,

in which area the highest number of loans was paid,

in which area the highest amount of loans was paid.

Select only **owners** of accounts as clients.

8) Client analysis - part 3

Use the query created in the previous task and modify it to determine the percentage of each district in the total amount of loans granted.

Sample result:

district_id	customer_amount	loans_given_amount	loans_given_count	amount_share
1	73	100	2	0.6666
74	17	50	5	0.3333

In other words, the task is to determine the distribution of granted loans by region.

9) Client selection

Check the database for the clients who meet the following results:

- their account balance is above 1000,
- they have more than 5 loans,
- they were born after 1990.

And we assume that the account balance is **loan amount - payments**.

10) Selection part 2

From the previous exercise you probably already know that there are no customers who meet the requirements. Make an analysis to determine which condition caused the empty results.

11) Expiring cards

Write a procedure to refresh the table you created (you can call it e.g. **cards_at_expiration**) containing the following columns:

- client_id,
- card_id,
- expiration_date - assume that the card can be active for 3 years after issue date,
- client_address (column **A3** is enough).

Note: The **card** table has cards that were issued until the end of 1998.

Determining the card's expiration date:

Suppose we have a card issued on **2020-01-01**, its expiration date according to the exercise conditions is **2023-01-01**. Because we want to mail new cards a week before the expiration date, then we just need to check the condition **2023-01-01 - 7 days = 2022-12-25 <= DATE <= 2023-01-01**.